1. Introduction

This plan created to give an overview about the SauceDemo website that is used to simulates a real online shopping system, and to practice software testing concepts, by letting users log in, browse item, manage carts, and complete the checkout process.

The plan will include testing objective, strategy and approach, scope, test environment, roles and responsibilities, risks, entry and exit criteria, testing timeline.

2. Testing Objective and Scope

The primary objective is to evaluate the functionality and stability of the SauceDemo website. By verifying the website's proper behavior according to normal valid inputs or abnormal invalid inputs, identifying defects, maintaining consistent performance, and ensuring the website meets the intended requirements.

The secondary objective is to detect all found defects and to report them separately in a corresponding bug report that is backed up by proof so it can be handled properly.

2.1 In-Scope:

1. User Roles:
    - Standard User
    - Problem User
    - Error User
    - Visual User
    - Locked Out User
    - Performance Glitch User
2. Features:
    - User Authentication (Log In, Log Out)
    - Item Browsing (price, description, image)
    - Item Sorting and Filtering
    - Add Items to Cart
    - Remove Items from Cart

- Reset App State Functionality
- Website's Response Time.
- Checkout Fields Validation (FirstName, LastName, ZipCode)
- Checkout Process and Confirmation.

2.2 Out-Of-Scope:

1. Features:
    - Website Security
    - Database Logic
    - Cross-Browser Compatibility
    - Payment Gateway Validation.

3. Test Strategy

Describes the overall approach, templates, and levels of testing to be performed.

3.1 Test Levels

- Unit Testing:
    - Authentication as Login, Logout
    - Cart Management (Add or Remove Items, Cart Counter)
    - Display Items' Information
    - Filter / Sort Items Functionality
    - Input Field Validation
    - Button Action Handling
- Integration Testing:
    - User Login or Logout Session that Redirects User
    - Filter and Sort that Updates Item Display
    - Reset App Stats that Updates Item Display
    - Add Items or Remove Items that Updates Cart Items and Counter
    - Validate Fields Input that Trigger Checkout flow
    - Checkout Validates Cart Contents before Proceeding

- System Testing:
    - Login → Browse Items → Filter Items → Add to Cart → Checkout → Order Confirmation → Logout.

3.2 Test Types

- Performance Testing
    - Login Response Time
    - Button Handling Response Time
- Functional Testing:
    - Verify Logging in with Valid Credentials
    - Verify Adding and Removing Items from Cart
    - Verify Complete Checkout with Valid Information
- Smoke Testing:
    - Validate User Login and Logout
    - Verify Items are listed and displayed
- Regression Testing:
    - Verify Item Sorting Correctly after UI Change
    - Verify Cart Icon Change after Adding or Removing Item
    - Verify User Session Still Up After Refresh

3.2 Test Approach

- **Manual Testing**

  It was performed using the traditional manual approach to validate critical workflows as authentication, cart management, item filtering, browsing, checkout process, and error handling. This approach has validated both expected (happy) and unexpected (negative) paths scenarios.

- **Test Artifacts:**
    - **Test Cases:** All were documented in a separate Excel Sheet, clearly stating id, description, test steps, test data, actual and expected results, status, evidence, notes if any.

- **Bug Reports:** Any defects found during testing phase were directly logged in a separate Excel Sheet with all needed details as id, summery, environment details, severity, priority, …etc.

- **Test Scenarios (Mapped to features):**

ID	Feature	Scenario Description	Type
TS-01	Login	Login with valid username = “standard_user” and password = “secret_sauce”	Happy
TS-02	Login	Login with invalid username= ”TestUser” and password= “secret_sauce”	Negative
TS-03	Item Browsing	View Item list after successful login	Happy
TS-04	View Item Description	Displaying another item’s description instead of one that is clicked	Negative
TS-05	Item Sorting	Click on Sort Items by price (low to high)	Happy
TS-06	Cart	Click on Add to Cart button, but the item is not added	Negative
TS-07	Cart	Click on Add Item to cart, and item is added	Happy
TS-08	Cart	Verify cart counter updates correctly after adding item	Happy
TS-09	Cart	Click on Remove Item from cart, and item is removed	Happy
TS-10	Cart	Attempt to remove item that is not in cart	Negative
TS-11	Checkout	Complete checkout with valid user information	Happy
TS-12	Checkout	Attempt checkout with missing user information	Negative
TS-13	Checkout	Attempt to go directly to confirmation page URL	Negative
TS-14	Checkout	Proceed to checkout with an empty cart	Negative
TS-15	Logout	Logout from the application successfully	Happy

ID	Feature	Scenario Description	Type
TS-16	Main Page Navigation	Attempt to access main page without being logged in	Negative

- **Automation:**

  In parallel with manual testing, most critical test cases were automated for end-to-end system testing to be time efficient and to reduce manual regression efforts. These Automated tests focused on frequently executed scenarios such login, logout, add to cart, remove from cart, checkout process, …etc. As well As, these test cases were automated for both **Standard User**, **Problem User**, covering both negative and happy path to verify the website's consistent behavior across test runs.

- **Test Design and Execution:**
  - Automated test cases were grouped in a test suite to allow selective execution based on user type.
  - Invocation Count is applied to specific test cases to execute them multiple times.
  - A Base Class was used to store shared data, allowing test classes to inherit from it, promoting maintainability.
  - Assertions are implemented in each test case to validate its outcome.

4. Test Environment and Tools

4.1 Environment

- **Operating System:** Windows 11.
- **Browser:** Google Chrome.
- **Website:** SauceDemo (https://www.saucedemo.com/)

4.2 Tools

- **Automation Platform:** Eclipse IDE.
- **UI Automation:** Selenium Framework, TestNG.
- **Evidence Management:** ShareX.

- **Test Management:** Excel Sheets.

5. Entry and Exit Criteria

5.1 Entry Criteria:

- All test platforms and tools must be successfully installed.
- A Stable internet connection must be established.
- All required test data must be available.
- Necessary documentation as test cases, bug reports, test scenarios should be reviewed and approved.

5.2 Exit Criteria:

- All test scenarios and test cases have been executed.
- All defects have been logged with evidences.
- All automated assertions have to be validated as expected.

5.3 Sign-Off:

- Project Manager
- QA Team Lead
- Development Lead

6. Assumptions

- SauceDemo Documentation will be given to the tester.
- Required Tools will be installed as Selenium, Eclipse, ShareX, … etc.
- Websites' behavior for valid users will be documented.
- Websites' behavior remains stable while testing.
- User's credentials as password and username are provided.

7. Test Design

7.1 Test Data

- Pre-Defined User Accounts
  - standard_user
  - problem_user

    - locked_out_user
    - visual_user
    - error_user
    - performance_glitch_user
- Website Items
- Checkout Information
    - First Name
    - Last Name
    - Zip Code

7.2 Techniques

- **State Transition:**

  Not logged in → Logging In → Logged In → Browse Items → Add Items to Cart → Start Checkout Process → Fill Required Information → Complete Checkout Process → Confirm Order.
- **Boundary Testing:**
    - Cart Quantity (Zero Item, One Item, Multiple Item).
    - Checkout Input Fields (First Name, Last Name, Zip Code)
        - Empty Input
        - String Input
- **Decision tables →** User Role Permissions.

7.3 Sample Test Case

- Role Based Scenario: locked_out_uesr can't log in.
- State Scenario: User can add items after successful login.
- Order confirmation page is displayed after successful checkout.
- Negative Scenario: Checking out without filling in required fields show validation errors.
- Regrade Updates: remove cart items updates the total price and cart icon number correctly.

- Boundary Scenario: website handles long inputs without crashing.
- Edge Scenario: adding and removing items repeatedly.

8. Defect Management Process

- **Defect Workflow:**
  - New → Assigned → Open → Fixed → Retest → Closed
- **Severity levels:**
  - High → Verify Checkout with Empty Cart.
  - Medium → Item Sorting and Filtering Malfunction
  - Low → UI Issues as Verify Cart Icon Visibility on different pages.
- **Defect Report Components:**
  - Description
  - Steps to Reproduce
  - Expected and Actual result,
  - Evidence
  - Severity and Priority
  - Environment

9. Metrics and Reporting

- **Metrics**
  - Number of manual, automated test cases and features covered.
  - Defect identified by severity and priority.
  - Percentage of passed automated test cases.
  - Time needed to run automation suite.
- **Reporting cadence →** daily during execution.

10. Schedule and Resources

- **Resources:**
  - A Project Manager
    - Acts as a primary contact for development and QA team.

    - Responsible for Project schedule and the overall success of the project
  - 1 Test Lead
    - Approve test plan, test cases, and automation methodology.
    - Monitor testing progress and report back to project manager.
  - 2 QA Engineers
    - Understand requirements and prepare test plan.
    - Write and execute test cases in excel sheet.
    - Identify Bugs and document them with evidence in excel sheet.
    - Design automated test cases and implement assertions.
    - Execute selenium automated results and analyze results.
- **Schedule example (12 working days):**
  - Day 1–2: Environment Setup + Installation.
  - Day 3–6: Planning + Test Cases Preparation + Bug Reports.
  - Day 7–9: Functional Testing
  - Day 9-11: Automation Design + Execute Selenium Automated Tests.
  - Day 12: Retest + final reporting/sign-off.

11. Risks and Mitigation

- Application downtime or instability → Perform Testing when Application is Available.
- Environment Issues as (network issue, browser version, …etc.) → Establish a Stable Internet Connection and State the Supported Browsers for SauceDemo.
- Incomplete Understanding of the Website's Requirements → Validate test scenarios across application behavior.
- Lack of Website Documentation → Perform Exploratory Testing to understand application behavior.
- Website Stability Over Time → Re-execute test cases after a specific amount of time (15 minutes).